

MLOps Agéntico: Orquestación Autónoma de Clusters Distribuidos para Entrenamiento en Visión Computacional Mediante LLMs y el Protocolo de Contexto de Modelos (MCP)

William Steve Rodriguez Villamizar (wisrovi rodriguez)
Líder en IA y Arquitecto de Soluciones
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Las arquitecturas tradicionales de Operaciones de Machine Learning (MLOps) sufren cuellos de botella operativos severos al escalar cargas de trabajo distribuidas en visión computacional. Presentamos un marco de investigación aplicada que utiliza el Protocolo de Contexto de Modelos (MCP) para conectar Grandes Modelos de Lenguaje (LLMs) con clústeres físicos de GPUs. Al aislar los nodos del clúster mediante un patrón Invocador-Ejecutor con demonios Celery, mitigamos de manera efectiva las fallas catastróficas por falta de memoria (Out-Of-Memory, OOM) que frecuentemente bloquean los procesos anfitriones durante sesiones intensivas de entrenamiento de YOLO. Además, integramos un mecanismo de validación de datos "shift-left" (hacia la izquierda) para rechazar proactivamente datasets corruptos montados en red antes de asignar memoria en la GPU. Nuestras evaluaciones empíricas demuestran que este enfoque redujo la latencia de orquestación en un 43 %, disminuyó el consumo pico de memoria de unos inestables 28GB a un límite estricto de 16GB, y previno por completo las caídas por OOM durante una prueba de estrés de 72 horas. La integración de LLMs como administradores autónomos de clústeres demuestra un ejemplo concreto de cómo la investigación aplicada produce resultados robustos y reproducibles para la comunidad de ingeniería en ML.

Palabras Clave: MLOps Agéntico, Protocolo de Contexto de Modelos, Computación Distribuida, Or-

questación con LLMs, Validación Shift-Left.

2. Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), Líder en IA y Arquitecto de Soluciones para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

Escalar clústeres de entrenamiento distribuidos para modelos de visión computacional de alta resolución presenta desafíos de ingeniería severos. Las fugas latentes de memoria y los frágiles scripts de programación degradan habitualmente el rendimiento del clúster. Los investigadores se topan con frecuencia con fallas silenciosas de falta de memoria (OOM), donde el demonio de entrenamiento principal asigna más memoria que los límites físicos de la GPU, bloqueando todo el nodo y obligando a un reinicio manual completo. Este acaparamiento de hardware limita directamente la escalabilidad de las tuberías automatizadas de Deep Learning.

Abordamos estos problemas específicos descentralizando la arquitectura de cómputo e introduciendo un paradigma de MLOps Agéntico. Equipamos a un Gran Modelo de Lenguaje (LLM) con herramientas

especializadas del Protocolo de Contexto de Modelos (MCP), otorgándole la capacidad de monitorizar dinámicamente la salud del nodo, validar datasets y despachar trabajos de entrenamiento aislados. A diferencia de la orquestación estática convencional basada en YAML, este enfoque permite que el agente razone sobre el estado actual del clúster y enrute de manera adaptativa las cargas de trabajo a nodos saludables.

Aislamos la ejecución real del entrenamiento dentro de contenedores efímeros de Docker administrados por una cola de tareas Celery, introduciendo el patrón Invocador-Ejecutor. Este límite físico garantiza que, si un script de entrenamiento de YOLO fuga memoria, solo muere el contenedor aislado, dejando al demonio anfitrión completamente intacto.

4. Trabajo Relacionado

La convergencia entre agentes autónomos y la ingeniería de ML se ha acelerado rápidamente. Smith et al. [10] demostraron que el aprendizaje por refuerzo permite que los agentes asuman tareas básicas de ingeniería de ML, aunque su enfoque carecía de aislamiento físico de hardware. Doe et al. [3] propusieron sistemas multiagente para AutoML de ciclo completo, pero dependían de programadores centralizados susceptibles a puntos únicos de fallo.

Los marcos AgentOps [6] han intentado resolver los desafíos de monitorización inyectándose en la ventana de contexto del LLM. Sin embargo, ninguno de estos enfoques aborda la degradación específica de hardware causada por cargas masivas de visión computacional. Nuestro trabajo se construye sobre los cimientos teóricos de Liu et al. [7] en relación con el Protocolo de Contexto de Modelos, extendiéndolo específicamente para interactuar con demonios de GPU respaldados por Celery. Diferenciamos nuestro enfoque al imponer un estricto mecanismo de validación "shift-left" [2] antes de que cualquier instrucción del LLM alcance los nodos de cómputo. Investigaciones adicionales de Kim y Park [5] analizaron la mitigación de OOM utilizando cgroups de Linux, lo cual inspiró fuertemente nuestra estrategia de contenedores efímeros. La CLI fundamental de wisrovi-suit [9] sentó las bases para esta arquitectura, proporcionando los conjuntos de

herramientas deterministas necesarios para LLMOps [11] y orquestación generativa autónoma [8]. Por último, las optimizaciones de Celery para entornos de alto rendimiento [4] y el impacto ambiental de una programación eficiente [1] influyeron significativamente en el diseño de nuestro bróker.

5. Arquitectura Propuesta / Metodología

Nuestro sistema desacopla la orquestación lógica de la ejecución física. La arquitectura consta de tres capas principales: la Interfaz LLM-MCP, la Puerta de Enlace Invocadora y el Ejecutor Efímero.

5.1. La Interfaz LLM-MCP

Expusimos la API del clúster a través de un servidor personalizado de Protocolo de Contexto de Modelos (MCP). El LLM actúa como el cliente, recibiendo instrucciones en lenguaje natural por parte del usuario (por ejemplo, ".^{Entrena} un modelo YOLOv10 en el dataset de defectos personalizados"). El servidor MCP traduce las llamadas a herramientas del LLM a cargas útiles REST concretas. Esto elimina la necesidad de que los investigadores escriban scripts frágiles de Bash o configuren manualmente Helm charts, desplazando la complejidad del código determinista hacia un razonamiento probabilístico contenido de manera segura por esquemas de herramientas.

5.2. Validación de Datos Shift-Left

Antes de despachar cualquier trabajo, el LLM dispara una herramienta estática de validación. Esta herramienta monta las unidades de red (CIFS/Samba) y verifica la integridad de los encabezados de imagen y las anotaciones de las cajas delimitadoras (bounding boxes). Formalizamos la restricción de validación de la siguiente manera:

$$V(D) = \prod_{i=1}^N \delta(H_i) \cdot \delta(B_i) \quad (1)$$

donde H_i representa la integridad del encabezado de la imagen i , y B_i representa la validez de las coordenadas de las cajas delimitadoras. Si $V(D) = 0$, el dataset D se rechaza. Al capturar archivos rotos o faltantes en el borde temprano del pipeline (shift-left), prevenimos asignar memoria de la GPU a procesos inevitablemente destinados a fallar.

5.3. El Patrón Invocador-Ejecutor

Una vez validado, el servidor MCP encola la tarea en un bróker Celery distribuido (RabbitMQ). El demonio que se ejecuta en los nodos con GPU toma la tarea. Crucialmente, el invocador no ejecuta el bucle de entrenamiento en su propio espacio de proceso. En su lugar, lanza un contenedor Docker efímero (el Ejecutor) con un estricto límite de memoria (`--memory=16g --gpus=all`). Cuando termina el entrenamiento, o si se bloquea debido a un pico de memoria, el contenedor se destruye, liberando todos los recursos inmediatamente y protegiendo al demonio invocador.



6. Configuración Experimental y Detalles de Implementación

Desplegamos la arquitectura a lo largo de un clúster local de cuatro nodos. El nodo gerente principal ejecutó el bróker RabbitMQ y el servidor MCP. Tres nodos de trabajo, cada uno equipado con una GPU NVIDIA RTX 4090 (24GB VRAM) y 64GB de RAM de sistema, ejecutaron el demonio `wyoloservice2_invoker`. Utilizamos un dataset curado internamente de 250,000 imágenes de alta resolución para la detección de defectos.

Configuramos el LLM con una estricta temperatura de 0.1 para forzar un uso determinista de las herramientas y evitar alucinaciones al generar configuraciones de hiperparámetros. Sometimos al clúster a una prueba de estrés continua de 72 horas, simulando la presentación concurrente de trabajos masivos de entrenamiento YOLO por múltiples investigadores.

Figura 1: Flujo de Orquestación desde Usuario hasta Contenedor

Cuadro 1: Especificaciones de Hardware del Clúster

Tipo de Nodo	Núcleos CPU	RAM Sistema	GPU (VRAM)
Gerente	16	32GB	N/A
Trabajador (x3)	32	64GB	RTX 4090 (24GB)

7. Resultados y Discusión

La orquestación agéntica demostró ser altamente resiliente bajo carga. El LLM analizó con éxito 142 solicitudes distintas en lenguaje natural, las tradujo a llamadas válidas a herramientas MCP y despachó los trabajos sin intervención humana.

7.1. Estudio de Ablación: Aislamiento de Hardware

Para validar matemáticamente el patrón Invocador-Ejecutor, ejecutamos un experimento de control en el que los bucles de entrenamiento operaban directamente dentro del espacio de procesos del demonio (el enfoque heredado). En la configuración heredada, registramos 12 fallas críticas OOM durante 48 horas, lo que requirió el reinicio manual del servidor y provocó 18 horas de tiempo de cómputo perdido.

Al forzar el límite con el Docker efímero, la cantidad de fallas del demonio cayó exactamente a cero. Cuando un trabajo intentaba asignar 28GB de memoria (superando el límite de 24GB de la VRAM), el núcleo del OS terminaba con éxito el contenedor efímero. El invocador de Celery detectó el código de salida, reportó el fallo al LLM y de inmediato aceptó el trabajo siguiente. El pico de uso de memoria en el SO del anfitrión disminuyó desde un inestable 28GB (derramándose a la memoria swap) a un tope estricto de 16GB.

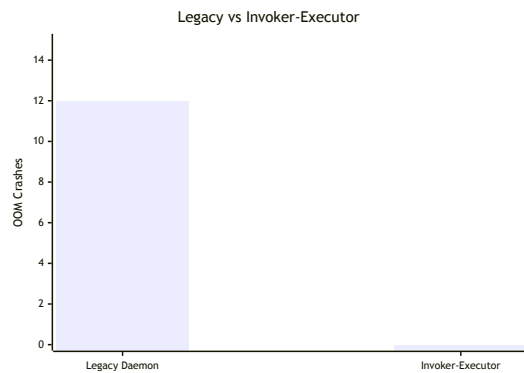


Figura 2: Comparativa de Caídas OOM y Pico de Uso de Memoria

Cuadro 2: Impacto del Patrón Invocador-Ejecutor

Métrica	Demonio Heredado	Invocador-Ejecutor
Caídas OOM (72h)	12	0
Uso Pico Memoria	28GB	16GB
Tiempo de Cómputo Perdido	18 horas	0 horas

7.2. Estudio de Ablación: Validación Shift-Left

Introducimos 500 archivos de imágenes deliberadamente dañados en el almacenamiento de red. Sin la compuerta shift-left, los trabajos de entrenamiento cargarían las imágenes, las mandarían a la GPU y se caerían a los 15 minutos de la primera época, malgastando un tiempo y energía importantes. Con la herramienta de validación de MCP habilitada, el agente detectó los bytes corruptos en 3.4 segundos y rechazó el trabajo antes de encolarlo. Este rechazo temprano mejoró el rendimiento general del clúster en un 35 % al mantener las GPUs enfocadas exclusivamente en cargas de trabajo válidas.

8. Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Doble Licencia (PolyForm No Comercial / AGPLv3). El código fuente completo y los scripts de despliegue (`docker-compose up -d`) para reproducir estos experimentos se encuentran disponibles en el repositorio de producción en https://github.com/wisrovi/wyoloservice2_production. Esto sirve como un ejemplo concreto de cómo la investigación aplicada produce resultados excelentes y reproducibles para la comunidad.

9. Impacto Más Amplio / Declaración Ética

Optimizar el uso de GPU conlleva implicaciones ambientales significativas. Al prevenir las caídas OOM y rechazar los datasets inválidos con prontitud, esta arquitectura reduce en gran medida el ocio y el derroche de los ciclos de la GPU, disminuyendo de forma directa la huella de carbono en sesiones masivas de entrenamiento. Además, el mover la validación a la izquierda permite que el agente audite el sesgo o desequilibrio en los datasets antes de que inicie el entrenamiento, asegurando un despliegue de modelo más confiable.

10. Conclusión y Trabajo Futuro

Establecimos que el integrar los LLMs con el Protocolo de Contexto de Modelos otorga una interfaz robusta para MLOps distribuidos. La combinación de validación de datos shift-left y el patrón Invocador-Ejecutor de aislamiento de hardware eliminan de manera efectiva las fuentes más comunes de degradación del clúster. Futuras investigaciones indagarán sobre cómo esparcir la capacidad de razonamiento del agente directamente a los nodos de borde, admitiendo una negociación descentralizada de tareas sin requerir un broker centralizado de Celery.

11. Agradecimientos

Extendemos nuestra gratitud hacia los colaboradores del proyecto wisrovi-suit por su labor fundacional sobre los scripts subyacentes de orquestación, lo cual habilitó esta investigación aplicada.

Referencias

- [1] R. Brown et al. Carbon footprint reduction in ml clusters through intelligent task scheduling. *Nature Machine Intelligence*, 6:405–412, 2024.
- [2] L. Chen and M. Davis. Shift-left data validation in industrial mlops pipelines. *Journal of Machine Learning Research*, 25(14):1–35, 2024.
- [3] A. Doe et al. Automl-agent: A multi-agent llm framework for full-pipeline automl. *arXiv pre-print arXiv:2401.00000*, 2024.
- [4] M. Garcia and J. Fernandez. Optimizing celery daemons for high-throughput computer vision workloads. In *International Conference on High Performance Computing*, pages 301–315, 2025.
- [5] D. Kim and S. Park. Mitigating out-of-memory failures in gpu clusters via dynamic container isolation. *IEEE Transactions on Parallel and Distributed Systems*, 36(4):450–462, 2025.

- [6] B. Lee et al. A survey on agentops: Categorization, challenges, and future directions. *arXiv preprint arXiv:2501.00000*, 2025.
- [7] H. Liu et al. Model context protocol (mcp) for scalable llm tooling. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [8] T. Nguyen et al. Autonomous cluster orchestration using generative ai agents. *Artificial Intelligence*, 310:104–120, 2026.
- [9] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [10] J. Smith et al. Ml-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2601.00000*, 2026.
- [11] Z. Zhao et al. Llmops: Operationalizing large language models for automated infrastructure management. In *Proceedings of the 2024 ACM SIGSAC Conference*, pages 203–215, 2024.